

SRNN Code Audit: MATLAB SRNNModel2 vs PyTorch srnn-E-only

Scope: Side-by-side comparison of SRNNModel2('n_a_E', 3, 'n_b_E', 1) (MATLAB, well-tested) against SRNNConfig(n_a_E=3, n_a_I=0, n_b_E=1, n_b_I=0) (PyTorch srnn_cell.py).

Note: The current srnn-E-only preset has n_a_E=1. This audit assumes the fix to n_a_E=3.

Date: 2026-04-08

Table 1: Initial Conditions

State layout: $S = [a_E(:); a_I(:); b_E(:); b_I(:); x(:)]$

State Variable	MATLAB	PyTorch	Match?	Notes
a_E (SFA, E)	<code>zeros(n_E * n_a_E, 1)</code>	<code>zeros(batch, n_E * n_a_E)</code>	YES	Both start at zero
a_I (SFA, I)	<i>absent</i> (n_a_I=0)	<i>absent</i> (n_a_I=0)	YES	—
b_E (STD, E)	<code>ones(n_E, 1)</code>	<code>zeros(batch, n_E)</code>	NO	PyTorch starts fully depressed (0), MATLAB starts fully available (1). b=0 is non-physical.
b_I (STD, I)	<i>absent</i> (n_b_I=0)	<i>absent</i> (n_b_I=0)	YES	—
x (dendritic)	<code>0.1 * randn(n, 1)</code>	<code>zeros(batch, N)</code>	WARN	MATLAB uses small random perturbation; PyTorch starts at exact zero. Acceptable for training (gradients break symmetry), but affects fidelity comparison.

[!WARNING] **Critical:** $b_E = 0$ means all excitatory synapses start fully depressed. The STD equation $db/dt = (1-b)/\tau_{rec} - b \cdot r/\tau_{rel}$ will recover toward 1, but the initial transient is qualitatively wrong compared to the MATLAB model.

Table 2: Parameters

All values verified by running both codebases. MATLAB values from `model.get_params()` after `build()`. PyTorch values from inspecting SRNNCell with `SRNNConfig(n_a_E=3, n_a_I=0, n_b_E=1, n_b_I=0)`.

Network Architecture

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Total neurons	N	300	32 (default)	—	Expected: different scale for training. Not a bug.

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
E fraction	f	0.5	$N//2 \Rightarrow 0.5$	YES	
E neuron count	n_E	150	16	—	Derived from N and f
I neuron count	n_I	150	16	—	Derived
SFA timescales (E)	n_a_E	3	1 (preset)	NO	Preset s rnn-E-only has 1, should be 3
SFA timescales (I)	n_a_I	0	0	YES	
STD channels (E)	n_b_E	1	1	YES	
STD channels (I)	n_b_I	0	0	YES	
Dale's law	—	Built into W via RMT	dales=True, softplus enforcement	YES	Different mechanism, same constraint
Sparsity	α	1/3 (indegree=100, n=300)	0.5 (random mask)	—	Different construction; expected

Time Constants

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Dendritic τ	τ_d	0.1 s (scalar)	$\text{softplus}(\text{inv_softplus}(0.1)) = 0.1$ s	YES	PyTorch can be per-neuron if per_neuron=True
SFA τ (E)	τ_{a_E}	logspace(0.25, 10, 3) = [0.25, 1.581, 10.0]	linspace(0.25, 10, 3) = [0.25, 5.125, 10.0]	NO	Middle timescale differs 3.2× (1.58 vs 5.13). MATLAB uses log-spacing; PyTorch uses linear interpolation.
STD recovery τ	τ_{rec}	1.0 s	$\text{softplus}(\text{inv_softplus}(1.0)) = 1.0$ s	YES	
STD release τ	τ_{rel}	0.25 s	$\text{softplus}(\text{inv_softplus}(0.25)) = 0.25$ s	YES	

Adaptation Coupling

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
SFA coupling (E)	c_E	0.15/3 = 0.0500 (scalar, same for all k)	softplus(−3.0) = 0.0486 (per-timescale vector)	WARN	2.8% lower. PyTorch allows per-timescale c_E (more flexible). Total coupling: MATLAB 0.15 vs PyTorch 0.146.
SFA resting point (E)	c_0_E	0.0	0.0	YES	
SFA coupling (I)	c_I	0.05 (unused, n_a_I=0)	N/A	—	
SFA resting point (I)	c_0_I	0.0 (unused)	N/A	—	

Activation Function

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Function	ϕ	PiecewiseSigmoid	piecewise_sigmoid	YES	Same 5-piece function
Linear fraction	S_a / q_φ	0.9	0.9	YES	Mathematically equivalent: $\phi(x; S_c=0.35) = \phi(x-0.35; S_c=0)$. Verified numerically.
Center / threshold	S_c / a_0	S_c = 0.35 (inside φ)	a_0 = 0.35 (subtracted before φ, S_c=0.0)	YES	

Solver

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Solver type	—	ode45 (adaptive RK45) or fused (semi-implicit)	semi_implicit (fixed-step)	—	Different solver classes; expected
Outer step size	h	1/fs = 1/400 = 0.0025 s	0.04 s	WARN	PyTorch 16× larger outer step
Sub-steps per step	—	fused_substeps = 6	ode_unfolds = 6	YES	Same sub-stepping concept
Effective dt	dt	0.0025/6 = 0.000417 s	0.04/6 = 0.00667 s	WARN	PyTorch sub-step is 16× larger than MATLAB fused sub-step
Sampling rate	f_s	400 Hz	1/h = 25 Hz	—	Training uses coarser time resolution

Input / Readout

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Input weights	W_in	eye(n) (fixed identity)	randn(N, input_size) × 0.1 (learned)	—	Expected: PyTorch learns input mapping
Readout mode	—	firing_rate (default)	synaptic (b·r)	WARN	Different default readout. MATLAB plots r; PyTorch outputs b·r.

Weight Matrix

Parameter	Symbol	MATLAB	PyTorch	Match?	Notes
Construction	W	RMT (Harris 2023) with E/I statistics	randn(N,N) * sqrt(2/N) + Dale's softplus	—	Fundamentally different; expected for analysis vs training
Scaling	level_of_chaos	1.0	N/A (no explicit scaling)	—	
Spectral radius	R	~2.37 (theoretical)	Emergent from init + training	—	

Table 3: Equations

Dendritic Potential

	MATLAB	PyTorch
ODE	$\frac{dx_i}{dt} = \frac{-x_i + u_i + \sum_j w_{ij} b_j r_j}{\tau_d}$	Same
Semi-implicit	$x_{n+1} = \frac{x_n + \frac{dt}{\tau_d} (W(b_{n+1} \cdot r_n) + u)}{1 + \frac{dt}{\tau_d}}$	$x_{n+1} = \frac{x_n + \frac{dt}{\tau_d} (W(b_n \cdot r_n) + u)}{1 + \frac{dt}{\tau_d}}$
Difference	Uses updated b (from step 2) in x update	Uses old b in x update
Source	SRNNModel2.m L1251–1261	srnn_cell.py L456–459

Firing Rate

	MATLAB	PyTorch
Equation	$r_i = \phi(x_i^{eff})$ where	$r_i = \phi(x_i^{eff} - a_0)$ where
Threshold	$x_i^{eff} = x_i - c_E \sum_k a_{ik}$ Built into ϕ via S_c = 0.35	$x_i^{eff} = x_i - \sum_k c_{E,k} \cdot a_{ik}$ Explicit subtraction: a_0 = 0.35, S_c = 0.0
Equivalence	$\phi(z; S_c=0.35) \equiv \phi(z - 0.35; S_c=0)$	YES Verified numerically

Spike-Frequency Adaptation (SFA)

	MATLAB	PyTorch
ODE	$\frac{da_{ik}}{dt} = \frac{c_0 + r_i - a_{ik}}{\tau_{a,k}}$	$\frac{da_{ik}}{dt} = \frac{-a_{ik} + c_{0,k} + r_i}{\tau_{a,k}}$
Match	YES Same equation	
Semi-implicit	$a_{n+1} = \frac{a_n + \frac{dt}{\tau_a}(c_0 + r_n)}{1 + \frac{dt}{\tau_a}}$	$a_{n+1} = \frac{a_n + \frac{dt}{\tau_a}(c_0 + r_n)}{1 + \frac{dt}{\tau_a}}$
Match	YES Same	
c_E application	Scalar: $c_E \cdot \sum_k a_{ik}$ Uniform c => equivalent when all c_E,k equal	Per-timescale: $\sum_k c_{E,k} \cdot a_{ik}$ More flexible for training

Short-Term Depression (STD)

	MATLAB	PyTorch
ODE	$\frac{db_i}{dt} = \frac{1-b_i}{\tau_{rec}} - \frac{b_i r_i}{\tau_{rel}}$	Same
Match	YES	
Semi-implicit	$b_{n+1} = \frac{b_n + \frac{dt}{\tau_{rec}}}{1 + dt(\frac{1}{\tau_{rec}} + \frac{r_n}{\tau_{rel}})}$	Same
Match	YES	
Post-update clamp	b = max(0, min(1, b)) Safety clamp to [0, 1]	None No clamping

Semi-Implicit Update Order

Step	MATLAB (fused_step)	PyTorch (_step_semi_implicit)
1	Compute r from current state	Compute r from current state
2	Update a (semi-implicit)	Compute b·r and W·(b·r) using old b
3	Update b (semi-implicit), clamp	Update x using old b·r
4	Rebuild b vector with new b	Update a (semi-implicit)
5	Update x using new b	Update b (semi-implicit)

[!IMPORTANT] MATLAB updates b *before* x and uses the freshly updated b in the x recurrence. PyTorch computes everything from the old state, then updates all variables. This is a minor but systematic numerical difference that accumulates over time.

Discrepancy Summary

#	Issue	Severity	Description
D1	n_a_E preset value	High	s rnn-e-only preset has n_a_E=1, but MATLAB reference uses n_a_E=3. Change preset or add s rnn-e-only-3sfa variant.

#	Issue	Severity	Description
D2	b_E initial condition	High	PyTorch <code>init_state()</code> returns all zeros. For b (synaptic availability), the physical initial condition is b=1 (no depression). MATLAB correctly initializes <code>b_E = ones(n_E, 1)</code> .
D3	tau_a_E spacing	Medium	MATLAB: log-spaced <code>[0.25, 1.58, 10.0]</code> . PyTorch: linearly interpolated <code>[0.25, 5.13, 10.0]</code> . The middle timescale differs by 3.2x, changing multi-timescale SFA dynamics substantially.
D4	Semi-implicit update order	Low	MATLAB uses updated b in the x update; PyTorch uses old b. Same formula, different coupling. Accumulates error over time but each step's error is $O(dt^2)$.
D5	c_E initial value	Low	MATLAB: 0.0500, PyTorch: <code>softplus(-3.0) = 0.0486</code> . 2.8% relative difference. Acceptable for trainable parameters.
D6	b_E clamping	Low	MATLAB clamps <code>bin[0,1]</code> after semi-implicit update. PyTorch does not. Since b is physical (synaptic availability fraction), clamping prevents non-physical states during extreme transients.
D7	x initial condition	Low	MATLAB: <code>0.1 * randn(n, 1)</code> , PyTorch: zeros. Minor for training but affects reproducibility comparisons.
D8	Readout default	Info	MATLAB default readout is <code>firing_rate(r)</code> , PyTorch default is <code>synaptic(b·r)</code> . Not a bug — PyTorch uses synaptic output as features.

Recommendations

Must Fix (D1, D2)

1. **Add `n_a_E=3` preset** — Create `srnn-e-only-3sfa` or update `srnn-E-only` to `n_a_E=3`
2. **Fix `init_state()` for `b`** — Initialize `b_E` and `b_I` to 1.0, not 0.0

Should Fix (D3)

3. **Use log-spacing for `tau_a_E`** — Change `_get_tau_a_E()` to interpolate in log-space:

```
# Current (linear): lo + (hi - lo) * t
# Proposed (log):  exp(log(lo) + (log(hi) - log(lo)) * t)
log_lo = torch.log(lo)
log_hi = torch.log(hi)
return torch.exp(log_lo + (log_hi - log_lo) * t)
```

Consider Fixing (D4, D6)

4. **Update order** — Optionally reorder semi-implicit to update `b` before `x` (matches MATLAB)
 5. **Clamp `b`** — Add `b_E_new = b_E_new.clamp(0, 1)` after STD update
-

Side Note: `parameter_table.md` Typo

The existing MATLAB documentation at `Intersect-LNNs-SRNNs/Docs/SRNN_docs/parameter_table.md` lists $\tau_{rel} = 1/2$ s, but the code (`SRNNModel2.m` L33) and verified parameter extraction both show $\tau_{b_E_rel} = 0.25$ s. The table value should be corrected to $1/4$ s.